★ · ★ · △ · ★ · ★

ANALISI OOP

Asteroids

Esercizi di analisi orientata agli oggetti in C++

Come usare questo documento

Questo documento ti guida nell'analisi del gioco Asteroids usando i concetti della Programmazione Orientata agli Oggetti (OOP) che hai studiato. Non si tratta di scrivere il codice completo del gioco, ma di allenare il tuo pensiero da progettista: capire quali classi servono, cosa devono sapere e cosa devono fare.

Per ogni sezione troverai:

- Una breve descrizione della competenza OOP affrontata
- Il modello delle classi di Asteroids come punto di riferimento
- Domande aperte da rispondere con ragionamenti motivati
- Una traccia (hint) per orientarti se sei in difficoltà

Attenzione alle domande trabocchetto (segnalate con il simbolo ⚠): sono domande con una risposta intuitiva ma errata. Scopri perchè!

△ Asteroids in poche parole

Leggi attentamente: da questa descrizione nascono le classi

Asteroids è un gioco arcade spaziale in cui il giocatore controlla una navicella triangolare in uno spazio bidimensionale. Lo scopo è distruggere tutti gli asteroidi presenti sullo schermo sparando proiettili, senza essere colpito dagli asteroidi stessi nè dalla nave aliena che appare periodicamente.

La navicella può ruotare a sinistra e a destra, attivare il propulsore per accelerare nella direzione in cui punta, e sparare proiettili. Non si ferma mai istantaneamente: continua a scorrere per inerzia nello spazio anche dopo aver smesso di accelerare. Lo spazio di gioco è toroidale: un oggetto che esce dal bordo destro rientra dal bordo sinistro, e analogamente per i bordi superiore e inferiore.

Quando un proiettile (o la navicella) colpisce un asteroide grande/medio/piccolo, questo viene distrutto. A seconda della dimensione la distruzione dell'asteroide vale un numero diverso di punti. Periodicamente appare una nave aliena che si muove e spara in modo autonomo: distruggerla vale molti punti.

Obiettivo

Pilotare la navicella spaziale all'interno di uno spazio toroidale, distruggendo tutti gli asteroidi presenti sullo schermo sparando proiettili, evitando le collisioni con gli asteroidi, con la nave aliena che appare periodicamente e con i suoi proiettili.

Elementi di Asteroids

Oggetto	Punti	Note
Asteroide grande	20 pt	Distrutto completamente dal primo proiettile
Asteroide medio	50 pt	Distrutto completamente dal primo proiettile
Asteroide piccolo	100 pt	Distrutto completamente dal primo proiettile
Nave aliena grande	200 pt	Appare periodicamente, spara a caso
Nave aliena piccola	1000 pt	Appare meno spesso, mira al giocatore

Azioni

- Ruota a sinistra / Ruota a destra — modifica l'angolo di orientamento della navicella senza cambiarne la velocità.
- Propulsore (Thrust) — applica una spinta nella direzione in cui punta la navicella; la velocità si accumula per inerzia e non si annulla quando si smette di premere.
- Spara — lancia un proiettile nella direzione in cui punta la navicella; il proiettile viaggia in linea retta e scompare dopo un tempo prefissato o al primo contatto con un asteroide o la nave aliena.
- Iperspazio (opzionale) — teletrasporta la navicella in una posizione casuale dell'arena; comporta un rischio di distruzione immediata.

Glossario Minimo

Termine	Significato
Arena	Lo spazio di gioco definito da larghezza e altezza; applica il wrapping toroidale a qualsiasi posizione che superi i propri bordi.
Spazio toroidale	Arena senza bordi reali: un oggetto che esce dal lato destro rientra da sinistra, uno che esce dall'alto rientra dal basso.
Navicella	Il vettore controllato dal giocatore; ha posizione, velocità, angolo e un numero limitato di vite.
Asteroide	Oggetto inerte che si muove in linea retta a velocità costante; esiste in tre taglie (grande, medio, piccolo) e viene distrutto completamente al primo colpo.
Proiettile	Oggetto sparato dalla navicella; viaggia in linea retta e ha una durata di vita limitata.
Nave aliena	Entità nemica che appare periodicamente, si muove lungo traiettorie irregolari e spara proiettili verso il giocatore.
Inerzia	La navicella mantiene la propria velocità anche quando il propulsore è spento; rallenta solo se viene applicata una spinta contraria.
Collisione	Contatto tra due oggetti che provoca la distruzione di entrambi (o di uno solo, nel caso proiettile-asteroide).

Responsabilità delle Classi

Ogni classe ha una responsabilità precisa. La tabella mostra cosa contiene e cosa sa fare ciascuna.

Classe	Contiene	Sa fare
Arena	Larghezza Altezza	Applicare il wrapping toroidale a una posizione
OggSpaziale	Posizione (x, y) Velocità (Vx, Vy) Angolo di orientamento Raggio come dimensione Stato: attivo / distrutto	Dire se due oggetti sono in collisione Aggiornare la posizione in base a Arena Rilevare la propria distruzione Dire il proprio raggio di collisione Visualizzare se stesso sullo schermo
Navicella	Numero di vite rimanenti Stato: vulnerabile / invulnerabile OggSpaziale	Applicare la spinta del propulsore Ruotare di un dato angolo Generare un Proiettile
Asteroide	Taglia: GRANDE / MEDIO / PICCOLO OggSpaziale	Dire la propria taglia
Proiettile	Timer di vita residua OggSpaziale	Sapere se è ancora attivo (timer > 0)
NaveAliena	Tipo: GRANDE / PICCOLA	Decidere quando e dove sparare

Classe	Contiene	Sa fare
	Timer di comparsa OggSpaziale	Generare un Proiettile nemico
GameEngine	Arena Navicella Array di Asteroidi Array di Proiettili NaveAliena Punteggio Stato del gioco: livello, esito	Avviare un nuovo livello Aggiornare tutte le entità a ogni frame Rilevare e risolvere tutte le collisioni tramite Arena Determinare la fine del livello o della partita
VistaTestuale	Campo Tastiera LettoreInput	Disegnare tutte le entità a schermo Leggere i comandi del giocatore Mostrare punteggio, vite e messaggi di esito

Relazioni tra le Classi

Tabella delle relazioni

Due tipi di relazione:

- **Possiede**: l'oggetto B fa parte di A e viene distrutto insieme a lui.
- **Usa**: A ha temporaneamente bisogno di B per svolgere un'operazione..

Chi	Relazione	Con chi	Nota
Navicella	**Possiede** e **Usa**	OggSpaziale	Navicella possiede il proprio OggSpaziale: posizione, velocità, angolo e raggio fanno parte della navicella e vengono distrutti con essa. Navicella usa OggSpaziale per muoversi e determinare le collisioni
Navicella	**Possiede** e **Usa**	Proiettile	Navicella possiede dei Proiettile e lo usa quando spara(); dopo o sparo il proiettile viene passato aa GameEngine che lo gestisce.
Asteroide	**Possiede** e **Usa**	OggSpaziale	Asteroide possiede il proprio OggSpaziale: posizione, velocità e raggio sono attributi dell'asteroide e vengono distrutti con lui. Asteroide usa OggSpaziale per muoversi e determinare le collisioni
Proiettile	**Possiede** e **Usa**	OggSpaziale	Proiettile possiede il proprio OggSpaziale: posizione e velocità sono dati interni al proiettile e vengono distrutti con lui. Proiettile usa OggSpaziale per muoversi e determinare le collisioni
NaveAliena	**Possiede** e **Usa**	OggSpaziale	NaveAliena possiede il proprio OggSpaziale: posizione, velocità e raggio fanno parte della nave aliena e vengono distrutti con essa. NaveAliena usa OggSpaziale per muoversi e determinare le collisioni

Chi	Relazione	Con chi	Nota
NaveAliena	**Possiede** e **Usa**	Proiettile	Navicella possiede dei Proiettile e lo usa quando spara(); dopo o sparo il proiettile viene passato aa GameEngine che lo gestisce
GameEngine	**Possiede** e **Usa**	Arena	GameEngine possiede Arena: le dimensioni del campo di gioco fanno parte del motore e vengono create e distrutte con esso. GameEngine usa Arena per calcolare le posizioni e comunicarle ai vari elementi
GameEngine	**Possiede** e **Usa**	Navicella	GameEngine possiede la Navicella del giocatore: è responsabile della sua creazione a inizio partita e della sua gestione per tutta la durata del gioco. GameEngine usa la Navicella per visualizzarla, spostarla e determinare se viene colpita da un Asteroide/ Proiettile/ NaveAliena
GameEngine	**Possiede** e **Usa**	Asteroide[]	GameEngine possiede tutti gli Asteroidi attivi: li crea a inizio livello e li rimuove quando vengono distrutti. GameEngine usa Asteroide per visualizzarlo, spostarlo e determinare se viene colpito da un Asteroide/ Proiettile/ NaveAliena/ Navicella
GameEngine	**Possiede** e **Usa**	Proiettile[]	GameEngine possiede tutti i Proiettili attivi (sia del giocatore sia nemici): li riceve da Navicella e NaveAliena e li rimuove quando vengono distrutti. GameEngine usa Proiettile per visualizzarlo, spostarlo e determinare se colpisce un Asteroide/ Proiettile/ NaveAliena/ Navicella
GameEngine	**Possiede** e **Usa**	NaveAliena	GameEngine possiede la NaveAliena: gestisce la sua comparsa periodica e la sua eventuale distruzione. GameEngine usa NaveAliena per visualizzarla, spostarla e determinare se colpisce un Asteroide/ Proiettile/ Navicella
IOTestuale	**Possiede** e **Usa**	Campo	IOTestuale possiede un campo da gioco e lo usa per visualizzare l'output su console
IOTestuale	**Possiede** e **Usa**	Tastiera	IOTestuale possiede una Tastiera e la usa per leggere i tasti direzionali premuti dal giocatore
IOTestuale	**Possiede** e **Usa**	LettoreInput	IOTestuale possiede LettoreInput e lo usa per chiedere il nome del giocatore
GameEngine	**Usa**	IOTestuale	GameEngine usa IOTestuale per leggere le indicazioni dell'utente o per visualizzare informazioni
OggSpaziale	**Usa**	IOTestuale	OggSpaziale usa IOTestuale per visualizzare il suo contenuto

Ciclo di un livello (5 passi)

1. Inizio livello: GameEngine crea l'Arena con le dimensioni della finestra, inizializza la Navicella al centro e genera un numero prefissato di Asteroidi — di taglia e velocità casuali — in posizioni lontane dalla navicella. GameEngine usa VistaTestuale per mostrare lo stato iniziale e attendere il primo input.
2. Ciclo di aggiornamento (game loop): a ogni frame GameEngine legge i comandi da VistaTestuale e li applica alla Navicella (rotazione, propulsore, sparo). Aggiorna poi la posizione di ogni entità attiva — Navicella, Asteroidi, Proiettili, NaveAliena — passando loro Arena affinché ciascuna applichi il wrapping toroidale in autonomia.
3. Rilevamento collisioni: GameEngine chiede ad Arena di verificare tutte le coppie rilevanti — Proiettile/Asteroide, Proiettile/NaveAliena, Navicella/Asteroide, Navicella/NaveAliena, Navicella/ProiettileNemico. Per ogni collisione rilevata rimuove le entità coinvolte e aggiorna il punteggio; se la collisione coinvolge la Navicella scala una vita e la riposiziona al centro.
4. Gestione NaveAliena: un timer controlla la comparsa periodica della NaveAliena. Una volta attiva, aggiorna la propria posizione tramite Arena e decide autonomamente quando sparare; GameEngine aggiunge il proiettile nemico al proprio array e lo gestisce come gli altri.
5. Esito: se tutti gli Asteroidi sono stati distrutti il livello è completato e GameEngine genera un nuovo insieme di Asteroidi più numerosi e veloci. Se la Navicella esaurisce le vite la partita termina. In entrambi i casi VistaTestuale mostra il messaggio di esito e il punteggio finale.

Scelte di Progetto

Perché tre classi per l'input/output?

Nella versione attuale il gioco funziona sul terminale: Campo usa cout per disegnare le entità con caratteri ASCII, Tastiera usa cin per leggere i comandi direzionali e di fuoco. Potrebbe sembrare eccessivo usare tre classi per qualcosa di così semplice.

La ragione è la seguente: se in futuro si vorrà aggiungere la grafica con SFML, sarà necessario sostituire la stampa su terminale con il rendering di una finestra e la gestione degli eventi. Se queste istruzioni fossero sparse nel codice, bisognerebbe cercarle e riscriverle ovunque.

Con questa architettura invece:

- **Campo** — definisce dove e come si disegna qualcosa (oggi: terminale con caratteri ASCII, domani: finestra SFML con sprite e rotazioni).
- **Tastiera** — definisce come si legge un comando (oggi: cin, domani: sf::Event di SFML con gestione del tasto tenuto premuto per il propulsore).
- **LettoreInput** — coordina la visualizzazione di un prompt e la raccolta della risposta. Se utilizza Campo e Tastiera non cambia, indipendentemente dalla tecnologia usata.

Quando si aggiungerà la grafica, GameEngine, Arena, Navicella, Asteroide, Proiettile e NaveAliena non cambieranno di una riga. Si riscriveranno solo Campo e Tastiera, perché sono loro gli unici a sapere come funziona l'output.

Perché GameEngine conosce tutto?

GameEngine possiede Arena, la Navicella, gli Asteroidi, i Proiettili e la NaveAliena ed è l'unico a gestire il ciclo di aggiornamento e il rilevamento delle collisioni. Questo lo rende una classe con molte responsabilità.

È una scelta consapevole per questa fase del progetto: permette di avere un unico punto di controllo facile da seguire e da debuggare. In Asteroids le collisioni coinvolgono coppie di entità che appartengono ad array diversi — solo chi li possiede tutti può verificarle in modo sistematico delegando il calcolo geometrico ad Arena. Quando il progetto crescerà, si potrà estrarre da GameEngine una classe CollisionManager dedicata. Riconoscere quel momento è una delle lezioni che si imparano sviluppando.

1 Classe — Il Progetto

Saper distinguere tra il modello astratto e l'entità concreta

Una classe è lo stampo, non l'oggetto. Asteroide è la classe; il grosso masso che si muove verso il centro dello schermo all'inizio del livello è l'oggetto (l'istanza). Quando progetti, devi sempre chiederti: "Sto descrivendo una categoria di cose, o una cosa specifica?"

Domanda 1 *[Classe]*

Nel modello proposto esiste un'unica classe Asteroide con un attributo dimensione (1=piccolo, 2=medio, 3=grande). Uno studente propone invece di creare tre classi separate: AsteroidePiccolo, AsteroideGrande, AsteroideMedio. Dal punto di vista della progettazione OOP questa scelta è corretta? Cosa si perde e cosa si potrebbe fare invece per gestire i comportamenti diversi legati alla dimensione?

Traccia: *Cosa hanno in comune i tre tipi di asteroide (posizione, velocità, collisione)? Cosa li differenzia? La differenza sta nella struttura o solo nei valori degli attributi (raggio, punti, numero di frammenti)?*

Domanda 2 *[Classe]*

Nel modello non compare una classe chiamata Arena che rappresenta lo spazio di gioco. Secondo te andrebbe rimossa trasferendo i suoi dati (larghezza e altezza) in GameEngine? Descrivi se la sua assenza è un problema concreto per gestire, ad esempio, il comportamento toroidale degli oggetti.

Traccia: *Il comportamento toroidale (uscire da un lato e rientrare dall'altro) deve essere applicato a tutti gli oggetti spaziali. Anche in GameEngine la posizione e la gestione è centralizzata, ma quali vantaggi ci sono ?*

Domanda 3 *[Classe]*

Un tuo compagno sostiene che Navicella e NaveAliena sono la stessa cosa e che basta una sola classe Nave. Ha ragione? Elenca almeno due differenze di stato o di comportamento che giustificano due classi distinte.

Traccia: *Guarda il modello: la Navicella ha vite e invulnerabile, la NaveAliena ha livelloMira. Pensa anche ai metodi: quali azioni può fare solo la Navicella del giocatore? Quali solo la NaveAliena?*

2 Attributo — Lo Stato

Saper identificare quali dati definiscono l'identità di un oggetto

Gli attributi definiscono cosa un oggetto sa di se stesso. La domanda giusta non è 'dove posso mettere questo dato?' ma 'a chi appartiene questa informazione per natura?'

Domanda 4 *[Attributo]*

La classe Navicella ha un attributo invulnerabile di tipo bool. A quale evento del gioco corrisponde (quando diventa true e quando torna false)? Scrivi la dichiarazione C++ dell'attributo e spiega perché è utile avere questo stato separato rispetto a gestirlo solo tramite le vite.

Traccia: *Dopo aver perso una vita, la navicella ricompare al centro dello schermo: se fosse subito vulnerabile, verrebbe colpita immediatamente dagli asteroidi vicini. Quanto dura l'invulnerabilità? Chi decide quando termina?*

Domanda 5 *[Attributo]*

La classe OggSpaziale ha sia posizione che velocità come Vettore2D. Perché è necessario mantenere entrambi separatamente? Non basterebbe aggiornare la posizione direttamente senza memorizzare la velocità? Collegalo al comportamento a inerzia della navicella.

Traccia: *La velocità non è solo 'quanta strada faccio in una iterazione': è uno stato che persiste tra un frame e l'altro e che si accumula. Se non la memorizzi, come sai di quante 'unità' stai ancora scorrendo dopo aver smesso di accelerare?*

Domanda 6 *[Attributo]*

La classe Asteroide ha un attributo dimensione di tipo int (1, 2, 3). Un altro studente propone di usare una stringa: "piccolo", "medio", "grande". Ci sono vantaggi e svantaggi in entrambe le scelte? Quale preferiresti nel codice C++ del gioco e perché?

Traccia: *Cosa succede quando devi confrontare la dimensione per decidere come frammentarsi? E quando devi calcolare i punti in base alla dimensione? Con quale tipo è più semplice fare un confronto o un calcolo?*

Domanda 7 *[Attributo]*

La classe Proiettile ha un attributo tempoVita di tipo int. A cosa serve? Dove viene decrementato? Cosa succede se non lo avessimo e i proiettili non scadessero mai?

Traccia: *Quanti proiettili potrebbe sparare il giocatore in un minuto di gioco? Se nessuno scade, quanti oggetti attivi ci sarebbero in memoria? E se un proiettile esce dallo schermo e rientra dall'altro lato, cosa potrebbe colpire?*

3 Metodo — Il Comportamento

Valutare la capacità di ripartire le azioni in blocchi logici

Un metodo è una responsabilità ben definita. Deve fare una cosa sola, farla bene, e il suo nome deve dire esattamente cosa fa. Se per spiegare un metodo usi la parola 'è, probabilmente dovrebbe essere diviso in due.

Domanda 8 *[Metodo]*

Descrivi il comportamento del metodo frammenta() nella classe Asteroide. Cosa deve restituire? Quali attributi del nuovo oggetto Asteroide devono essere inizializzati e con quali valori? Chi si occupa di aggiungere i nuovi frammenti al gioco dopo che frammenta() è stato chiamato?

Traccia: *Il metodo restituisce due nuovi Asteroidi. Chi li riceve? Chi li aggiunge alla lista degli oggetti attivi? Il metodo frammenta() dovrebbe aggiungere se stesso alla lista, o dovrebbe limitarsi a creare e restituire i frammenti?*

Domanda 9 *[Metodo]*

Il metodo aggiorna() compare sia in OggSpaziale, sia in Asteroide, sia in Proiettile, sia in NaveAliena. Tutti e quattro fanno la stessa cosa? Descrivi cosa fa aggiorna() per almeno due di queste classi e spiega perché ha senso che ogni classe abbia la propria versione del metodo.

Traccia: *aggiorna() di OggSpaziale calcola la nuova posizione in base alla velocità. aggiorna() di Proiettile fa lo stesso ma in più decrementa tempoVita. aggiorna() di NaveAliena può includere anche il calcolo della traiettoria di movimento autonomo. Cosa cambia?*

Domanda 10 *[Metodo]*

Un metodo chiamato gestisciTurno() in una ipotetica classe GestoreGioco fa queste cose: aggiorna tutti gli oggetti, controlla tutte le collisioni, rimuove gli oggetti non attivi, genera nuovi asteroidi se lo schermo è vuoto, aggiorna il punteggio e ridisegna lo schermo. È un buon metodo? Identificane i problemi e proponi come suddividerlo.

Traccia: *Conta quante responsabilità diverse ha questo metodo. Quali sono logica di gioco e quali sono logica di visualizzazione? Quali potrebbero diventare metodi privati separati chiamati da gestisciTurno()?*

Domanda 11 *[Metodo]*

Il metodo collideWith(o:OggSpaziale) di OggSpaziale controlla se due oggetti si sovrappongono usando i loro raggi. Questo metodo dovrebbe stare in OggSpaziale oppure in una classe esterna GestoreCollisioni? Confronta le due scelte.

Traccia: *Se sta in OggSpaziale, l'oggetto sa da solo se è stato colpito. Se sta in GestoreCollisioni, nessun oggetto conosce gli altri. Quale delle due rende più facile aggiungere nuovi tipi di oggetti in futuro?*

4 Incapsulamento — Protezione e Interfaccia

Verificare se si sa proteggere l'integrità dei dati

Incapsulare non significa solo mettere private davanti a tutto. Significa decidere consapevolmente quale parte del tuo oggetto è visibile all'esterno (l'interfaccia pubblica) e quale è un dettaglio interno che nessun altro deve toccare direttamente.

Domanda 12 *[Incapsulamento]*

Nella classe OggSpaziale, posizione e velocità sono private. Un tuo compagno dice: 'basterebbe renderle public, così il codice del gioco può aggiornare posizione e velocità direttamente senza passare per i metodi'. Spiega almeno due problemi concreti che nascerebbero in Asteroids se si seguisse questo consiglio.

Traccia: *Se velocità fosse public, chi garantisce che il wrap toroidale (uscire da un lato e rientrare dall'altro) venga applicato alla posizione ogni volta che cambia? E se due parti del codice modificassero la velocità contemporaneamente in modo incoerente?*

Domanda 13 *[Incapsulamento]*

La classe Navicella espone il metodo perdiVita() come pubblico. Questo significa che chiunque può far perdere una vita alla navicella in qualsiasi momento. È corretto? Chi dovrebbe avere il diritto di chiamare questo metodo, e come progetteresti l'interfaccia per limitarne l'uso?

Traccia: *Solo una collisione con un asteroide o un proiettile nemico dovrebbe causare la perdita di una vita. Ma in C++ non puoi limitare 'chi' chiama un metodo pubblico. Come si gestisce convenzionalmente questo problema di progettazione?*

⚠ **Domanda 14 trabocchetto** *[Incapsulamento]*

La classe Asteroide ha il metodo frammenta() che crea e restituisce due nuovi asteroidi. Se rendessimo pubblico l'attributo dimensione (invece di avere getDimensione()), il codice esterno potrebbe modificarlo direttamente — ad esempio scrivere asteroide.dimensione = 0. Quali conseguenze avrebbe nel gioco?

Traccia: *Cosa succede se la dimensione diventa 0 o un valore negativo? Il metodo frammenta() usa dimensione per calcolare la dimensione dei figli: cosa restituisce se dimensione vale 0? L'oggetto diventerebbe inconsistente.*

5 Composizione — Relazione "Ha un"

Valutare la capacità di costruire oggetti complessi assemblando pezzi semplici

La composizione è il modo principale in cui gli oggetti collaborano: un oggetto contiene un altro oggetto come attributo. Navicella HA UN OggSpaziale. Asteroide HA UN OggSpaziale. Proiettile HA UN OggSpaziale. Costruire bene la composizione significa evitare oggetti troppo grandi e responsabilità duplicate.

Domanda 15 *[Composizione]*

Navicella, Asteroide, Proiettile e NaveAliena contengono tutti un attributo OggSpaziale di tipo OggSpaziale. Questo significa che la logica di posizione, velocità e collisione è duplicata quattro volte? Oppure è la scelta corretta? Giustifica la tua risposta usando il concetto di composizione.

Traccia: *Duplicare codice e duplicare la struttura sono due cose diverse. Tutte e quattro le classi usano OggSpaziale, ma chi implementa la logica di movimento e collisione? Quante volte è scritta quella logica nel codice?*

Domanda 16 *[Composizione]*

Se volessi aggiungere un power-up (es. uno Scudo temporaneo che appare nello spazio e che la navicella può raccogliere), progetteresti una nuova classe Scudo con un proprio OggSpaziale, oppure riutilizzeresti una classe esistente aggiungendo attributi? Motiva la scelta.

Traccia: *Lo Scudo ha una posizione, una velocità (magari nulla), un raggio di collisione e uno stato attivo/non attivo. Esiste già nel modello una classe con esattamente queste caratteristiche? Cosa cambia aggiungendo solo il comportamento specifico?*

⚠ **Domanda 17 trabocchetto** *[Composizione]*

La classe Vettore2D è usata sia per posizione che per velocità in OggSpaziale. Uno studente dice: 'basterebbe usare quattro double separati: posX, posY, velX, velY, senza creare una classe Vettore2D'. Quali vantaggi perde con questa scelta?

Traccia: *Il metodo aggiorna() calcola posizione = posizione + velocità. Con Vettore2D questa operazione è una riga. Con quattro double separati, quante righe diventano? E se aggiungi la rotazione, che richiede operazioni trigonometriche su entrambe le componenti?*

6 Responsabilità — Chi fa cosa

Capire se si sa evitare la creazione di 'Classi Diò che fanno tutto

Ogni classe deve avere una sola ragione di esistere e una responsabilità chiara. Se una classe conosce troppo degli altri, o fa cose che non le competono, diventa fragile: ogni modifica al gioco richiede di toccarla. La domanda da farsi sempre è: 'Questa azione appartiene davvero a questa classe?'

Domanda 18 *[Responsabilità]*

Immagina che uno studente crei una sola classe AsteroidsGame che gestisce tutto: gli oggetti spaziali, le collisioni, il punteggio, la generazione di nuovi asteroidi, la grafica e l'input da tastiera. Quali sono i problemi pratici di questa scelta quando il gioco deve essere modificato (es. aggiungere un nuovo tipo di nemico o cambiare le regole di frammentazione)?

Traccia: *Prova a pensare a cosa devi toccare nella classe AsteroidsGame ogni volta che cambia anche solo una piccola regola. E se vuoi riusare la logica degli asteroidi in un gioco diverso?*

Domanda 19 *[Responsabilità]*

La logica di frammentazione (un asteroide grande diventa due medi, un medio diventa due piccoli) si trova nel metodo frammenta() della classe Asteroide. Questa logica potrebbe stare anche in una classe esterna RegolaFragmentazione. Quale delle due posizioni ti sembra più corretta e perché? Ci sono argomenti validi per entrambe?

Traccia: *Chi possiede la regola di frammentazione? È una caratteristica intrinseca di ogni asteroide (sa come frammentarsi) o è una regola esterna del gioco che viene applicata agli asteroidi dall'esterno?*

⚠ **Domanda 20 trabocchetto** *[Responsabilità]*

Il metodo aggiorna() di OggSpaziale aggiorna la posizione in base alla velocità. Dovrebbe anche occuparsi del comportamento toroidale (se esci a destra rientri a sinistra)? Oppure questa responsabilità appartiene a qualcun altro? Proponi una soluzione e discutine i pro e i contro.

Traccia: *Se il wrap toroidale è dentro OggSpaziale, ogni oggetto gestisce autonomamente i propri confini. Ma OggSpaziale dovrebbe conoscere le dimensioni dello schermo di gioco. È corretto che un oggetto fisico conosca i limiti del campo da gioco?*

Domanda 21 *[Responsabilità]*

Scrivi la dichiarazione C++ (solo l'intestazione, non l'implementazione) di una classe GestoreOndataAsteroidi che abbia la responsabilità di creare e gestire le ondate di asteroidi per ogni livello. Quali attributi deve avere? Quali metodi? Giustifica ogni scelta.

Traccia: *Una classe GestoreOndataAsteroidi non deve implementare la logica interna degli asteroidi: deve coordinarli. Quanti asteroidi ci sono in ogni ondata? Come aumenta la difficoltà tra un livello e l'altro? Chi la chiama e quando?*

Riepilogo delle competenze

```
1  Classe           |  Saper distinguere il modello astratto dall'entità
concreta
2  Attributo        |  Identificare i dati che definiscono l'identità di
un oggetto
3  Metodo           |  Ripartire le azioni in blocchi logici con
responsabilità chiara
4  Incapsulamento   |  Proteggere l'integrità dei dati e progettare
l'interfaccia
5  Composizione     |  Costruire oggetti complessi assemblando pezzi
semplici
6  Responsabilità   |  Distribuire bene le responsabilità, evitare le
Classi Dio
```